

Технічна документація

ВСТУП

Технічне завдання. Розробити об'єктно-орієнтовану систему керування магазином товарів, яка дозволяє зареєстрованим користувачам робити замовлення товарів різних типів та надсилати SMS повідомлення у випадку рекламних акцій та днів народжень. Паролі не зберігати у відкритому вигляді. Персональні дані користувача не повинні бути доступні іншим особам. Для збереження інформації використати текстові файли власного формату даних. Забезпечити розподілений доступ на основі ролей (Administrator, Advanced user, User). Кожна операція має логуватись так, щоб за журналом можна було відновити базу даних. Запропонувати варіант захисту (шифрування/дешифрування) бази даних і логів. Забезпечити безпечну обробку вхідних даних і протестувати функціонал unit-тестами. Розмістити ПЗ на GitHub разом з документацією і UML-діаграмами.

Мета документа Надати повний технічний опис існуючої реалізації проекту, що дозволяє розробникам, тестувальникам і DevOps інженерам швидко зрозуміти архітектуру, інтерфейси, модель даних, процеси розгортання, спостереження, тестування і експлуатації без внесення змін у код.

Обсяг документа Документ охоплює бекенд, фронтенд і набір QA тестів. Описує структуру репозиторіїв, доступні конфігурації для локального запуску, перелік основних ендпоінтів, модель даних, механізми спостереження, логування, міграції, тестування і рекомендації з експлуатації.

Аудиторія Розробники, тестувальники, DevOps інженери, технічні менеджери і зацікавлені сторони.

Огляд системи

Короткий опис Проект складається з трьох взаємопов'язаних частин: бекенд на Python, фронтенд на Vite JavaScript і набір автоматизованих тестів на pytest. Для локального запуску використовується Docker Compose з профілями для мінімального стеку і для повного стеку зі спостереженням. Бекенд працює з PostgreSQL і Redis, має інтеграцію з OpenTelemetry, VictoriaLogs, VictoriaTraces і Grafana.

Загальна структура системи

Проект складається з трьох взаємопов'язаних репозиторіїв:

Репозиторій	Призначення	Основні технології
cakf-task1-backend	Серверна частина — REST API, бізнес-логіка, робота з даними, логування, спостереження	Python, FastAPI, PostgreSQL, Redis, Docker
cakf-task1-frontend	Клієнтська частина — SPA, маршрутизація, UI компоненти, інтеграція з API	JavaScript, Vite, React
qa-sdet-tests	Автоматизовані тести для API та інтеграції	Python, pytest

КОМПОНЕНТИ СИСТЕМИ

- **Backend** — REST API, бізнес логіка, міграції, health endpoint, автогенерація документації API.
- **Frontend** — SPA на Vite, маршрутизація, компоненти інтерфейсу, підключення до API.
- **QA tests** — автоматизовані тести для API і інтеграції.
- **Infrastructure** — Docker Compose, Postgres, Redis, OpenTelemetry collector, VictoriaLogs, VictoriaTraces, Grafana.

АРХІТЕКТУРА І РОЗГОРТАННЯ

Архітектурна схема

Короткий опис взаємодії Клієнтський додаток звертається до REST API бекенду. Бекенд зберігає дані в PostgreSQL і використовує Redis для кешування або черг. OpenTelemetry збирає трейси і логи, які експортуються у VictoriaLogs і VictoriaTraces. Grafana підключена до VictoriaLogs і VictoriaTraces для візуалізації.

Архітектура бекенду

Бекенд побудований за **багатошаровою архітектурою**, що розділяє логіку на рівні:

1. **Рівень маршрутів (routes)** — визначає REST-ендпоінти, приймає запити, викликає сервіси.

2. **Рівень сервісів (services)** — містить бізнес-логіку, обробку даних, перевірку прав доступу.
3. **Рівень репозиторіїв (repositories)** — відповідає за роботу з базою даних або файлами.
4. **Рівень моделей і схем (models, schemas)** — описує структуру даних, що зберігаються або передаються через API.
5. **Рівень залежностей (dependencies)** — визначає спільні компоненти, наприклад, підключення до БД, кешу, логування.
6. **Рівень утиліт (utils)** — допоміжні функції: шифрування, форматування, логування, робота з часом.

Структура папок

```
cakf-task1-backend/
├── app/
│   ├── routes/           # REST API маршрути (auth, users, orders, products)
│   ├── services/         # Бізнес-логіка (створення замовлень, перевірка ролей)
│   ├── repositories/     # Робота з даними (патерн Repository, CRUD)
│   ├── models/           # ORM моделі (User, Order, ProductType, AuditLog)
│   ├── schemas/          # Pydantic схеми для валідації запитів/відповідей
│   ├── dependencies/     # Підключення до БД, Redis, конфігурації
│   ├── utils/            # Допоміжні функції (шифрування, логування)
│   ├── middleware/       # Middleware для RBAC, логування, трасування
│   └── main.py            # Точка входу додатку
├── alembic/              # Міграції бази даних
├── deployment/           # Docker Compose, collector config, Grafana provisioning
├── tests/                # Unit та integration тести
├── pyproject.toml        # Залежності
├── uv.lock               # Lock-файл для пакетів
└── entrypoint.sh         # Скрипт запуску контейнера
```

```
cakf-task1-frontend/
├── src/
│   ├── app/              # Головний компонент і маршрутизація
│   ├── layouts/          # Макети сторінок (AppLayout)
│   ├── pages/            # Сторінки (Login, Register, Products, Orders)
│   ├── components/       # Повторно використовувані UI-компоненти
│   ├── services/         # API-запити до бекенду
│   ├── utils/            # Допоміжні функції (валідація, форматування)
│   └── index.jsx          # Точка входу
├── public/               # Статичні файли
├── package.json           # Залежності
├── vite.config.js         # Конфігурація Vite
└── index.html             # HTML-шаблон
```

```
qa-sdet-tests/
├── tests/
│   ├── test_auth.py      # Тести реєстрації та логіну
│   ├── test_orders.py    # Тести створення замовлень
│   ├── test_products.py  # Тести CRUD для товарів
│   └── conftest.py       # Фікстури для тестового клієнта
├── requirements.txt       # Залежності для тестування
└── pytest.ini            # Конфігурація pytest
```

Взаємодія компонентів

- **Маршрути (routes)** приймають HTTP-запити, викликають відповідні **сервіси (services)**.
- **Сервіси** реалізують бізнес-логіку, перевіряють права доступу, формують відповіді.
- **Сервіси** звертаються до **репозиторіїв (repositories)**, які виконують операції з базою даних через ORM.
- **Репозиторії** використовують **моделі (models)** для опису таблиць і **схеми (schemas)** для валідації даних.
- **Middleware** додає логування, трасування, RBAC-перевірки.
- **Utils** забезпечують шифрування, форматування, роботу з часом, маскування чутливих даних.
- **Dependencies** інjektують спільні ресурси (підключення до БД, Redis, конфігурації).

Патерн Repository

Робота з даними реалізована через **патерн Repository**, який відокремлює бізнес-логіку від доступу до даних.

Переваги:

- Легше тестувати сервіси без реальної бази (можна використовувати мок-репозиторії).
- Зміна джерела даних (Postgres → файли або інша БД) не потребує змін у бізнес-логіці.
- Код стає більш структурованим і підтримуваним.

Взаємодія між репозиторіями

1. Frontend → Backend

- Викликає REST API через HTTP.
- Використовує JWT для авторизації.
- Отримує JSON-відповіді з даними користувачів, товарів і замовлень.

2. Backend → QA Tests

- Тести перевіряють API бекенду.
- Використовують тестову базу даних або мок-сервіси.

3. Backend → Infrastructure

- Docker Compose піднімає всі сервіси (API, БД, Redis, Grafana).
- OpenTelemetry збирає метрики і логи.

Docker Compose профілі і сервіси

Профілі

- **app** — мінімальний стек для роботи додатку і бази даних.
- **obs** — повний стек з додатковими сервісами для спостереження.

Основні сервіси

- `api` або `api-obs` — контейнер бекенду.
- `db` — PostgreSQL.
- `redis` — Redis.
- `otel-collector` — OpenTelemetry collector.
- `victorialogs` — VictoriaLogs.
- `victoriatraces` — VictoriaTraces.
- `grafana` — Grafana.

Типові порти

- API 8000
- Grafana 3000
- VictoriaLogs 9428
- VictoriaTraces 10428
- Postgres 5432
- Redis 6379

КОМАНДИ ДЛЯ ЛОКАЛЬНОГО ЗАПУСКУ

Запуск мінімального стеку

bash

```
docker compose -f deployment/local/docker-compose.yaml --profile app up --build
```

Запуск повного стеку зі спостереженням

```
bash
```

```
docker compose -f deployment/local/docker-compose.yaml --profile obs up --build
```

Зупинка

```
bash
```

```
docker compose -f deployment/local/docker-compose.yaml --profile app down
```

API І КОНТРАКТИ

Загальні принципи

- **Формат:** REST JSON.
- **Аутентифікація:** JWT у заголовку Authorization: Bearer <token>.
- **Документація:** Swagger UI доступний локально за адресою /docs, ReDoc за /redoc.
- **Health check:** endpoint /api/v1/health/ для readiness і liveness перевірок.

Основні ендпоінти і поведінка

Auth

- POST /api/v1/auth/register — реєстрація користувача. Повертає статус створення і дані користувача без пароля.
- POST /api/v1/auth/login — логін, повертає access token.

Products

- GET /api/v1/products — отримати список товарів. Підтримка фільтрації і пагінації залежно від реалізації.
- POST /api/v1/products — створити товар. Доступно для ролей з правами створення.

Orders

- POST /api/v1/orders — створити замовлення. Валідація кількості і наявності товару.
- GET /api/v1/orders — отримати список замовлень для адміністратора або власника.

Users

- GET /api/v1/users/{id} — отримати профіль користувача. Доступно власнику або ролям з правами перегляду.

Формат помилок

Структурований JSON Помилки повертаються у структурованому форматі з полями для коду помилки, HTTP статусу, повідомлення і додаткових деталей. Кожна відповідь помилки має унікальний request_id для трасування.

МОДЕЛЬ ДАНИХ

Основні сутності

User

- **id** — унікальний ідентифікатор.
- **email** — унікальний контакт.
- **full_name** — повне ім'я.
- **birth_date** — дата народження.
- **delivery_address** — адреса доставки.
- **phone** — телефон.
- **role** — роль користувача.
- **password_hash** — хеш пароля.
- **created_at** — дата створення запису.

ProductType

- **id** — унікальний ідентифікатор.
- **name** — назва типу товару.
- **description** — опис.
- **price** — ціна.
- **available_quantity** — доступна кількість.

Order

- **id** — унікальний ідентифікатор.
- **user_id** — посилання на користувача.
- **product_type_id** — посилання на тип товару.
- **quantity** — кількість.

- **order_date** — дата замовлення.
- **status** — статус замовлення.

AuditLog

- **id** — унікальний ідентифікатор.
- **entity_type** — тип сутності.
- **entity_id** — ідентифікатор сутності.
- **action** — дія create update delete.
- **performed_by** — хто виконав дію.
- **timestamp** — час дії.
- **payload** — JSON з деталями зміни.

Міграції

У репозиторії присутня папка для міграцій бази даних. Для застосування змін у схемі використовуються стандартні інструменти міграцій, що дозволяють створювати і відкотити версії схеми.

БЕЗПЕКА І ЗАХИСТ ДАНИХ

Паролі і аутентифікація

- **Паролі** зберігаються у вигляді хешів.
- **JWT** використовується для аутентифікації і захисту маршрутів.

Захист персональних даних

- Чутливі поля маскуються у логах.
- Для експорту і видалення персональних даних передбачені механізми відповідно до вимог користувача.

Контроль доступу

- Використовується RBAC з ролями Administrator Advanced user User.
- Захищені маршрути перевіряють ролі і права доступу.

Рекомендації з безпеки експлуатації

- Зберігати секрети у захищеному сховищі, не комітити .env у репозиторій.
- Використовувати TLS у продакшн середовищі для всіх зовнішніх з'єднань.
- Регулярно оновлювати залежності і образи контейнерів.

СПОСТЕРЕЖЕННЯ І ЛОГУВАННЯ

OpenTelemetry і collector

- У deployment присутній конфіг для OpenTelemetry collector, який приймає OTLP HTTP і експортує логи і трейси у VictoriaLogs і VictoriaTraces. Collector монтується у контейнер і доступний на стандартному порту.

Логи

- Логи структуровані у форматі JSON з полями timestamp level service message request_id user_id path duration_ms extra.
- Чутливі поля маскуються перед відправкою у систему логів.

Метрики і дашборди

- Метрики запитів latency error rate request rate зберігаються у системі спостереження.
- Grafana підключена до джерел даних для візуалізації логів і трас.

ТЕСТУВАННЯ І ЯКІСТЬ

Автоматизовані тести

- **Unit tests** для бізнес логіки.
- **Integration tests** для перевірки API проти тестової бази даних.
- **E2E тестування** може бути реалізоване у окремому наборі тестів.

Запуск тестів

- Тести запускаються за допомогою pytest. Приклад команди для локального запуску:

```
bash
```

```
pytest --maxfail=1 --disable-warnings -q
```

- У CI тестове середовище піднімає тимчасові сервіси Postgres і Redis для ізоляції тестів.

Покриття і вимоги

- Рекомендується досягти високого покриття для критичних компонентів. тести повинні перевіряти сценарії реєстрації логіну створення замовлень і обробки помилок.

CI CD І РЕЛІЗИ

Рекомендована структура pipeline

- **Lint** — статичний аналіз коду.
- **Unit tests** — запуск pytest.
- **Build** — збірка Docker образів.
- **Integration tests** — підняття тестового стеку і запуск інтеграційних тестів.
- **Publish** — пуш образів у реєстр при успішному проходженні тестів.

Секрети і конфігурація

- Зберігати секрети у CI secret store.
- Міграції бази даних виконувати як окремий крок перед релізом.

РЕЗЕРВНЕ КОПІЮВАННЯ І ВІДНОВЛЕННЯ

Політика бекапів

- Регулярні повні дампи бази даних.
- Інкрементальні бекапи логів і важливих артефактів.
- Ротація і зберігання бекапів у захищеному зовнішньому сховищі.

Процедура відновлення

- Відновлення з останнього повного дампу.
- Реплей аудиту для відновлення останніх операцій, якщо це необхідно.
- Перевірка цілісності даних і запуск smoke тестів після відновлення.

ОПЕРАЦІЙНІ СЦЕНАРІЇ

Initial data load

- Імпорт початкових даних у базу через наявні механізми імпорту або через API.

Checkout

- Створення замовлення, оновлення кількості товару, запис аудиту.

Data correction

- Адміністратор вносить зміни через захищені ендпоінти, зміни фіксуються в журналі аудиту.

ДОКУМЕНТАЦІЯ ДЛЯ РОЗРОБНИКА

Структура бекенду

- `app/` — маршрути сервіси репозиторії схеми.
- `alembic/` — міграції.
- `deployment/` — Docker Compose і конфігурації.
- `main.py` — точка входу.

Структура фронтенду

- `src/` — компоненти сторінки роутер.
- `index.html package.json vite.config.js` — конфігурація.

Короткі інструкції

- Перед запуском створити локальний `.env` на основі прикладу.
- Піднімати стек через Docker Compose з потрібним профілем.
- Виконувати міграції перед запуском у середовищі, де це потрібно.